**UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA**

Faculty of Computing and Mathematical Sciences

Department of Computer Science and Engineering

# CE 473 – LOGIC OF COMPUTER SCIENCE

## COMPLETE EXAM STUDY GUIDE

*Every Example · Every Operation · Every Proof — Fully Worked*

Compiled from the Official UMaT Handout by Dr. W.A. Agangiba & Dr. A.A. Amponsah

January 2024 Edition

# TABLE OF CONTENTS


| | |
|---|---|
| Chapter 1 | Mathematical and Structural Induction |
| 1.1 | Mathematical Induction |
| 1.2 | Principles of Mathematical Induction – Examples & Proofs |
| 1.3 | Principles of Structural Induction – Examples & Proofs |
| Chapter 2 | Boolean Algebra |
| 2.1 | Overview & Postulates |
| 2.2 | Truth Tables |
| 2.3 | Logical Operators – All Truth Tables |
| 2.4 & 2.5 | Simplification of Boolean Functions – All Examples |
| 2.6 | Karnaugh Maps – All Examples |
| 2.7 | Logic Gates – All 7 Gates |
| Chapter 3 | Propositional Logic |
| 3.1 | Language of Propositional Logic |
| 3.2 | Truth Valuation |
| 3.3 | Satisfiability of Formulae |
| 3.4 | Tautologies, Contradictions, Contingencies |
| 3.5 | Logical Equivalence – All Examples & Proofs |
| 3.6 | Conjunctive Normal Form – All Examples |
| 3.7 | Natural Deductions – All Rules & Worked Examples |
| 3.8 | Limitations of Propositional Logic |
| Chapter 4 | Predicate Logic |
| 4.1 | Translation Key & Examples |
| 4.2 | Quantifiers and Variables |
| 4.3 | Syntax of Quantifiers |
| 4.4 | Syllogistic Statements |
| 4.5 | Conjunctive Predicate Combinations |
| 4.6 | Disjunctive Predicate Combinations |
| 4.7 | Laws and Rules of Inference – All Proofs |
| 4.8 | Indirect Proof |
| 4.9 | Conditional Proof |

# CHAPTER 1: MATHEMATICAL AND STRUCTURAL INDUCTION

## 1.1 Mathematical Induction

**Mathematical induction** is a method for proving statements whose domain is a subset of the integers.

It is useful for discovering and characterising regular patterns or sequences.

It is especially important in CS because most programs involve *repetition of sequences of statements*.

A statement proven by induction is typically a function of a natural number **n**, where n = 1, 2, 3, …

## 1.2 Principles of Mathematical Induction

To prove a statement holds for all positive integers **n**, follow **THREE steps**:

**The Three Steps of Mathematical Induction**

Step 1 – BASE CASE: Prove the statement is true for the smallest value of n

(usually n = 0, 1, or 2).

Step 2 – INDUCTION HYPOTHESIS (IH): Assume the statement is true for n = k

(some arbitrary fixed value, at least the base).

Step 3 – INDUCTIVE STEP: Prove the statement is true for n = k + 1,

using the assumption in Step 2.

■ This technique ONLY works when the domain is a subset of the integers. It CANNOT prove results for non-integer values (e.g. n = 2.5).

### Example 1: Show that $n^2 \geq 2n$ for all n = 2, 3, 4, …

**Worked Solution – Example 1**

STEP 1 – Base Case (n = 2):

LHS = $2^2 = 4$

RHS = 2(2) = 4

Since $4 \geq 4$, the statement holds for n = 2. ✔

STEP 2 – Induction Hypothesis (IH):

Assume $n^2 \geq 2n$ holds for n = k (where $k \geq 2$).

Therefore: $k^2 \geq 2k$ …(IH)

STEP 3 – Inductive Step: Prove for n = k + 1

Need to show: $(k+1)^2 \geq 2(k+1)$

LHS = $(k+1)^2 = k^2 + 2k + 1$

By IH, $k^2 \geq 2k$, so:

LHS = $k^2 + 2k + 1 \geq 2k + 2k + 1 = 4k + 1$

RHS = $2(k+1) = 2k + 2$

We need LHS ≥ RHS:

$k^2 + 2k + 1$ vs $2k + 2$

Since $k \geq 2$: $2k > 1$, therefore $2k + 2k + 1 > 2k + 2$

More directly: $k^2 + 2k + 1 \geq 2k + 2k + 1 > 2k + 2$ ✔

$\therefore (k+1)^2 \geq 2(k+1)$ is TRUE.

CONCLUSION: By the principle of mathematical induction, $n^2 \geq 2n$ for all $n \geq 2$. ■

## Example 2 (Extra): Prove that 1 + 2 + 3 + … + n = n(n+1)/2 for all n ≥ 1

### Worked Solution – Example 2 (Sum Formula)

STEP 1 – Base Case (n = 1):

LHS = 1

RHS = 1(1+1)/2 = 1

LHS = RHS ✔

STEP 2 – Induction Hypothesis:

Assume 1 + 2 + … + k = k(k+1)/2 for some k ≥ 1.

STEP 3 – Inductive Step: Prove for n = k + 1

1 + 2 + … + k + (k+1)

= k(k+1)/2 + (k+1) [using IH]

= (k+1)[ k/2 + 1 ]

= (k+1)(k + 2)/2

= (k+1)((k+1) + 1)/2 [this is exactly the formula for n = k+1] ✔

CONCLUSION: The sum formula holds for all n ≥ 1. ■

## Example 3 (Extra): Prove 2■ > n for all n ≥ 1

### Worked Solution – Example 3

STEP 1 – Base Case (n = 1):

LHS = $2^1 = 2$

RHS = 1

2 > 1 ✔

STEP 2 – IH: Assume 2■ > k for some k ≥ 1.

STEP 3 — Show 2^(k+1) > k + 1:

2^(k+1) = 2 · 2■

> 2 · k [by IH, 2■ > k]

= 2k

= k + k

≥ k + 1 [since k ≥ 1]

∴ 2^(k+1) > k + 1 ✔

CONCLUSION: 2■ > n for all n ≥ 1. ■

## 1.3 Principles of Structural Induction

**Structural induction** is used to prove a property P holds for all elements of a *recursively defined data type*.

Unlike mathematical induction (over integers), structural induction mirrors the recursive definition of the structure itself.

**Two Steps of Structural Induction**

Step 1 — BASE CASE: Prove P for the 'base cases' of the definition.

Step 2 — INDUCTIVE STEP: Prove P for the result of any combination rule,

ASSUMING it is true for all the parts (sub-structures).

### Recursive Definition Examples from the Handout

The handout gives three standard recursive definitions:

**Recursive Definition 1 – Even Numbers**

E is defined as:

1. 0 ∈ E (base case)

2. if n ∈ E then n + 2 ∈ E (combination)

3. if n ∈ E then -n ∈ E (combination)

Consequence: E = {…, -4, -2, 0, 2, 4, …} = the set of ALL even integers.

**Recursive Definition 2 – Non-empty Binary Tree**

A non-empty binary tree T is either:

1. A single root node r, OR

2. A root node r connected (by an edge) to the root(s) of one or two

non-empty binary trees TL and/or TR.

**Recursive Definition 3 – Fully Bracketed Arithmetic Expression**

A fully bracketed arithmetic expression E over {[, ], +, -, *, x} is either:

1. The symbol 0, 1, or x (base cases)

2. [e + e'] where e, e' are such expressions

3. [e * e'] where e, e' are such expressions

## Structural Induction Proof 1: |V| = |E| + 1 for any non-empty binary tree

Claim: For any non-empty binary tree T = (V, E), the number of vertices equals the number of edges plus one: |V| = |E| + 1

**Proof by Structural Induction on T = (V, E)**

Let P(T) be the property |V| = |E| + 1.

BASE CASE – T is a single root node:

|V| = 1 (just the root)

|E| = 0 (no edges)

1 = 0 + 1 ✔

INDUCTIVE STEP:

Suppose T = (V,E) has root r connected to:

• The root rL of tree TL = (VL, EL) [always present]

• Possibly the root rR of tree TR = (VR, ER)

Induction Hypothesis: P(TL) and P(TR) both hold, i.e.

|VL| = |EL| + 1 and |VR| = |ER| + 1

Case A – T has only TL (one child):

|V| = |VL| + 1 [r + all nodes of TL]

= (|EL| + 1) + 1 [by IH on TL]

= |EL| + 2

|E| = |EL| + 1 [edges of TL + edge (r, rL)]

∴ |V| = |EL| + 2 = (|EL| + 1) + 1 = |E| + 1 ✔

Case B – T has both TL and TR (two children):

|V| = |VL| + |VR| + 1

= (|EL|+1) + (|ER|+1) + 1 [by IH]

= |EL| + |ER| + 3

|E| = |EL| + |ER| + 2 [edges of TL + edges of TR + 2 new edges]

∴ |V| = |EL| + |ER| + 3 = (|EL| + |ER| + 2) + 1 = |E| + 1 ✔

In both cases P(T) holds. By structural induction, P(T) is true for EVERY non-empty binary tree T. ■

## Structural Induction Proof 2: Every element of E is of the form 2x, x ∈ ■

**Proof by Structural Induction on even number e**

Let P(e) be the property: e = 2x for some integer x ∈ ■.

BASE CASE — e = 0:

0 = 2 · 0 and 0 ∈ ■ ✔

INDUCTIVE STEP: Two cases from the recursive definition.

IH: Assume P(e') holds, i.e. e' = 2x' for some x' ∈ ■.

Case 1 — e = e' + 2:

e = e' + 2 = 2x' + 2 = 2(x' + 1) = 2x

where x = x' + 1 ∈ ■ (since x' ∈ ■) ✔

Case 2 — e = -e':

e = -e' = -(2x') = 2(-x') = 2x

where x = -x' ∈ ■ (since x' ∈ ■) ✔

Therefore, for every even number e, P(e) holds.

Hence E = {2x | x ∈ ■} (the set of all even integers). ■

# CHAPTER 2: BOOLEAN ALGEBRA

## 2.1 Overview & Postulates

**Boolean logic** forms the basis for computation in modern binary computers. Any algorithm or circuit can be represented using Boolean equations.

**Boolean algebra** is a deductive mathematical system closed over {0, 1} (False, True).

A *binary operator* accepts two Boolean inputs and produces one Boolean output.

The fundamental **postulates** of Boolean algebra are:

| ID | Law | Name |
|---|---|---|
| (a) | x + y = y + x | + is commutative |
| (b) | x · y = y · x | · is commutative |
| (c) | (x+y)+z = x+(y+z) | + is associative |
| (d) | (x·y)·z = x·(y·z) | · is associative |
| (e) | x+(y·z) = (x+y)·(x+z) | + distributes over · |
| (f) | x·(y+z) = (x·y)+(x·z) | · distributes over + |
| (g) | 0 + x = x | 0 is identity for + |
| (h) | 1 · x = x | 1 is identity for · |
| (i) | x + x■ = 1 | complement law for + |
| (j) | x · x■ = 0 | complement law for · |
| (k) | x + x = x | idempotent law for + |
| (l) | x · x = x | idempotent law for · |
| (m) | x·(x+y) = x | absorption law |
| (n) | x+(x·y) = x | absorption law |
| (o) | 0 · x = 0 | zero property |
| (p) | 1 + x = 1 | one property |

| | | |
|---|---|---|
| (q) | x■+■ = x■·■ | de Morgan's law |
| (r) | x■·■ = x■+■ | de Morgan's law |
| (s) | x■■ = x | double complement law |
| (t) | (x·y)+(x·■) = x | consensus |
| (u) | (x+y)·(x+■) = x | consensus |

## 2.2 Truth Tables

In Boolean logic only two values exist: **True (1)** and **False (0)**.

A truth table lists ALL possible combinations of input values and the resulting output.

For n variables, there are **2■ rows** in the truth table.

*Number of rows required*

| n vars | Rows needed |
|---|---|
| 1 | 2 |
| 2 | 4 |
| 3 | 8 |
| 4 | 16 |

## 2.3 Logical Operators – All Truth Tables

### 1. Conjunction (AND) — x ∧ y

Output is TRUE **only when both** inputs are TRUE.

| x | y | x ∧ y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Laws from AND:

(i) x∧y ≡ y∧x (commutative)

(ii) x∧1 ≡ x

(iii) x∧0 ≡ 0

(iv) x∧x ≡ x (idempotent)

### 2. Disjunction (OR) — x ∨ y

Output is FALSE **only when both** inputs are FALSE.

| x | y | x ∨ y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 1 | 1 |

Laws from OR:

(i) x∨y ≡ y∨x (commutative)

(ii) x∨1 ≡ 1

(iii) x∨0 ≡ x

(iv) x∨x ≡ x (idempotent)

### 3. Equivalence (XNOR) — x ↔ y

Output is TRUE when both inputs are **equal**.

| x | y | x ↔ y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

### 4. Implication — x → y

Output is FALSE **only when** x is TRUE and y is FALSE.

| x | y | x → y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

### 5. Negation / NOT — ¬x

Unary operator. Reverses the truth value.

| x | ¬x |
|---|---|
| 0 | 1 |
| 1 | 0 |

## 2.4 & 2.5 Simplification of Boolean Functions

Boolean simplification reduces circuits to equivalent ones with *fewer components*, increasing reliability and reducing cost.

The **algebraic method** applies the postulates/laws step by step.

### Example 1 (from handout): Simplify A ∨ A·B

**Solution**

```
A ∨ A·B

= A·(1 ∨ B) [factor out A]

= A·1 [Law of 1 for disjunction: 1∨B = 1]

= A [identity: A·1 = A]
```

Result: A ∨ A·B = A ✔

### Example 2 (from handout): Simplify (A ∨ B)(A ∨ C)

**Solution**

(A ∨ B)(A ∨ C)

= A·A ∨ A·C ∨ B·A ∨ B·C [distributive property, expand]

= A ∨ A·C ∨ B·A ∨ B·C [idempotent: A·A = A]

= A ∨ B·C [absorption: A∨A·C = A and A∨B·A = A]

Result: (A ∨ B)(A ∨ C) = A ∨ BC ✔

### Example 3 (Extra): Simplify AB + A■B + AB■

**Solution**

AB + A'B + AB'

= B(A + A') + AB' [factor B from first two terms]

= B·1 + AB' [complement law: A + A' = 1]

= B + AB' [identity: B·1 = B]

= B + A [absorption: B + AB' = B + A since B + B' = 1]

Verification by truth table (A=0,B=0: 0+0+0=0; A=0,B=1: 0+1+0=1;

A=1,B=0: 0+0+1=1; A=1,B=1: 1+0+0=1) → same as A+B ✔

## 2.6 Karnaugh Maps (K-Maps)

A **Karnaugh map (K-map)** is a pictorial method for simplifying Boolean expressions. It avoids the need to remember all algebraic laws.

Adjacent cells differ in **exactly one bit** (Gray code ordering).

Cells are grouped in powers of 2 (1, 2, 4, 8, …) to find prime implicants.

**Key Rules for Grouping:**

**✔ Groups must be rectangular and contain 2■ cells (i ≥ 1).**
**✔ Groups may wrap around edges (the map is a torus).**
**✔ Larger groups give simpler (fewer literals) product terms.**
**✔ Use the FEWEST groups to cover all 1-cells (for SOP) or 0-cells (for POS).**
■ A diagonal grouping or groups of 3, 5, 6, 7 are INVALID.

**Resulting Product Term from a Group:**

• If variable X = 0 for ALL cells in the group → include X' in the term.

• If variable X = 1 for ALL cells in the group → include X in the term.

• If variable X changes (0 and 1) within the group → X is ELIMINATED.

### Worked K-map Example 1 (SOP from handout, map X)

**Given map X — 4-variable (A,B,C,D):**

Reading the 1-cells and grouping (as shown in handout):

Group 1: A·B cells → product term: A·B

Group 2: A'·B'·D' cells → product term: A'·B'·D'

Group 3: A'·B'·C cells → product term: A'·B'·C

Group 4: B·C·D' cells → product term: B·C·D'

SOP Result: X = A·B + A'·B'·D' + A'·B'·C + B·C·D'

### Worked K-map Example 2 (SOP from handout, map Y)

**Given map Y:**

Group 1: A'·C' cells → product term: A'·C'

Group 2: A·C·D' cells → product term: A·C·D'

Group 3: A·B·C cells → product term: A·B·C

Group 4: B·C'·D' cells → product term: B·C'·D'

SOP Result: Y = A'·C' + A·C·D' + A·B·C + B·C'·D'

### Worked K-map Example 3 (POS from handout)

**Product-of-Sums: group the 0-cells instead of the 1-cells**

Given map Z (3-variable example from handout):

0-cells are at positions where the output = 0.

Group M3 and M7 (both zero cells):

M3·M7 = (A+B'+C')·(A'+B'+C')

= (A·A') + (B'+C') [by distribution]

= 0 + (B'+C')

= B' + C'

POS Result: Z = B' + C'

Verification from the example in the handout also shows:

Z = (A+D')·(B'+C+D)·(A'+B+C'+D)

## 2.7 Logic Gates – All Seven Basic Gates

### AND Gate

Output = 1 ONLY when BOTH inputs are 1. (Acts like logical AND)

| In 1 | In 2 | Out |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

### OR Gate

Output = 1 when AT LEAST ONE input is 1. (Acts like logical OR)

| In 1 | In 2 | Out |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

### XOR Gate (Exclusive-OR)

Output = 1 when inputs are DIFFERENT. Output = 0 when inputs are SAME.

| In 1 | In 2 | Out |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

### NOT Gate (Inverter)

Unary gate. Output is the COMPLEMENT of input.

| Input | Output |
|---|---|
| 0 | 1 |
| 1 | 0 |

### NAND Gate

AND followed by NOT. Output = 0 ONLY when both inputs are 1.

| In 1 | In 2 | Out |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

### NOR Gate

OR followed by NOT. Output = 1 ONLY when both inputs are 0.

| In 1 | In 2 | Out |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

### XNOR Gate (Exclusive-NOR)

XOR followed by NOT. Output = 1 when inputs are SAME.

| In 1 | In 2 | Out |
|---|---|---|
| 0 | 0 | 1 |

| | | |
|---|---|---|
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Arrays of logic gates form **Integrated Circuits (ICs)**. As IC technology advances, gates shrink, enabling more complex operations at higher speeds.

# CHAPTER 3: PROPOSITIONAL LOGIC

Propositional logic (also called **sentential/statement logic**) deals with logical relationships between propositions taken as wholes.

A **proposition** is a statement that is either TRUE or FALSE (never both, never neither).

Propositional logic is interested in how the truth value of *compound claims* depends on the truth values of individual claims.

**Proposition:** A declarative statement with a definite truth value (T or F).

**Non-proposition:** Statements with pronouns/variables like 'n is prime' or 'He is president' — not decidable until the variable is specified.

### Negation

Given proposition p, its negation ~p always has the OPPOSITE truth value.

Example: P2 = 'It is raining' → ~P2 = 'It is not raining'

### Connectives (Compound Propositions)

Simple propositions are combined using **connectives**:

**Simple and Compound Proposition Examples**

```
p1: It is raining.

p2: I am wet.

p3 = p1 ∧ p2 : It is raining AND I am wet. (conjunction)

p4 = p1 ∨ p2 : It is raining OR I am wet. (disjunction)

p5 = p1 ∧ ¬p2: It is raining AND I am NOT wet.
```

## 3.1 Language of Propositional Logic

The language Lv is built from:

1. Propositional variables (uppercase letters A, B, C, P, Q, R, …)

2. Five connectives: ¬ (NOT), ∨ (OR), ∧ (AND), → (IMPLIES), ↔ (IFF)

3. Parentheses ( and )

**Propositional Formula:** A well-formed expression built by the grammar:

• Any variable V is a formula.

• If F is a formula, so is (¬F).

• If F and G are formulas, so are (F∨G), (F∧G), (F→G), (F↔G).

### Formation Trees

A **formation tree** shows how a formula is built from its components. Bottom nodes are always propositional variables.

**Formation Tree Example: ((A⇒B)∧(B∨C)) ⇒ (A∨C)) ⇒ ¬(B∨D)**

```
The main connective is the outer ⇒.

Left sub-tree: ((A⇒B)∧(B∨C)) ⇒ (A∨C)

Right sub-tree: ¬(B∨D)
```

Abbreviated tree replaces each formula node with its main connective symbol.

Bottom nodes remain as variable letters (A, B, C, D).

### Subformulas

A **subformula** of formula P is any formula appearing as a node in P's formation tree.

**Subformulas of (P∧Q) ∨ ¬(P∧Q)**

1. (P∧Q) ∨ ¬(P∧Q) [the whole formula]
2. P∧Q
3. ¬(P∧Q)
4. P
5. Q

## 3.2 Truth Valuation

A **truth assignment** M maps each variable to T or F: M : V → {T, F}

The unique extension VM : FV → {T,F} satisfies:

**Valuation Rules**

1. VM(A) = M(A) [variables]
2. VM(¬P) = T iff VM(P) = F [negation]
3. VM(P∨Q) = T iff VM(P)=T or VM(Q)=T [disjunction]
4. VM(P∧Q) = T iff VM(P)=T and VM(Q)=T [conjunction]
5. VM(P→Q) = T iff VM(P)=F or VM(Q)=T [implication]
6. VM(P↔Q) = T iff VM(P) = VM(Q) [equivalence]

### Worked Valuation Example (from handout)

Formula P = ((A∧B) ↔ (A∨¬C))

**Case (a): M(A)=T, M(B)=T, M(C)=F**

VM(¬C) = T [since M(C)=F]

VM(A∨¬C) = T [T ∨ T = T]

VM(A∧B) = T [T ∧ T = T]

VM(P) = VM(T ↔ T) = T [same truth values] ✔

**Case (b): M(A)=T, M(B)=F, M(C)=T**

VM(¬C) = F [since M(C)=T]

VM(A∨¬C) = T [T ∨ F = T]

VM(A∧B) = F [T ∧ F = F]

VM(P) = VM(F ↔ T) = F [different truth values]

## 3.3 Satisfiability of Formulae

**Satisfiable:** Set S is satisfiable if there EXISTS a truth assignment M that makes every formula in S true simultaneously.

**Unsatisfiable:** No truth assignment makes all formulae in S true.

**Case (a) – Satisfiable Example**

$S = \{ (A \wedge B) \Rightarrow C,\ A \leftrightarrow B,\ (C \vee A) \vee B \}$

Try M: $M(A)=T,\ M(B)=T,\ M(C)=T$

$(A \wedge B) \Rightarrow C : T \wedge T \Rightarrow T = T \Rightarrow T = T$ ✔

$A \leftrightarrow B : T \leftrightarrow T = T$ ✔

$(C \vee A) \vee B : T \vee T \vee T = T$ ✔

All formulas are TRUE → S is SATISFIABLE. $M(A)=M(B)=M(C)=T$ satisfies S.

**Case (b) – Unsatisfiable Example**

$S = \{ A \Rightarrow (B \vee C),\ \neg(A \Rightarrow B),\ A \leftrightarrow \neg C \}$

Try $M(A)=T$:

$A \leftrightarrow \neg C$ requires $\neg C=T \rightarrow M(C)=F$

$A \Rightarrow (B \vee C)$: $T \Rightarrow (B \vee F) = T \Rightarrow B$, so we need $M(B)=T$

$\neg(A \Rightarrow B) : \neg(T \Rightarrow T) = \neg T = F \leftarrow$ CONTRADICTION

Try $M(A)=F$:

$\neg(A \Rightarrow B) : \neg(F \Rightarrow B) = \neg T = F \leftarrow$ CONTRADICTION (regardless of B)

No truth assignment makes all formulas true → S is UNSATISFIABLE.

## 3.4 Tautologies, Contradictions and Contingencies

**Tautology:** A formula that is TRUE under EVERY possible truth assignment.

**Contradiction:** A formula that is FALSE under EVERY possible truth assignment.

**Contingency:** A formula that is NEITHER a tautology nor a contradiction.

### Tautology Laws

If t is a tautology, c is a contradiction, p is any proposition:

**Tautology Laws**

$p \wedge t \equiv p$

$p \vee t \equiv t$ (NOT p, because: t is always TRUE, so $p \vee t$ is always TRUE)

$\neg t \equiv c$

**Contradiction Laws**

$p \wedge c \equiv c$

$p \vee c \equiv p$

¬c ≡ t

### Truth Table Verification: p ∨ ¬p (Tautology) and p ∧ ¬p (Contradiction)

*p∨¬p is always T (tautology); p∧¬p is always F (contradiction)*

| p | ¬p | p∨¬p | p∧¬p |
|---|---|---|---|
| T | F | T | F |
| F | T | T | F |

### Worked Example: Show p ∨ (q ∨ ¬p) is a Tautology

**Method 1 – Truth Table**

p | q | ¬p | q∨¬p | p∨(q∨¬p)

T | T | F | T | T

T | F | F | F | T

F | T | T | T | T

F | F | T | T | T

All rows give TRUE → it is a TAUTOLOGY. ✔

**Method 2 – Algebraic (from handout)**

p ∨ (q ∨ ¬p)

≡ p ∨ (¬p ∨ q) [commutative]

≡ (p ∨ ¬p) ∨ q [associative]

≡ t ∨ q [p∨¬p = tautology t]

≡ t [t∨q = t by domination]

Therefore p ∨ (q ∨ ¬p) is a tautology. ■

## 3.5 Logical Equivalence

Propositions r and s are **logically equivalent** (written r ≡ s or r ■ s) if and only if r ↔ s is a *tautology*, i.e. they have the SAME truth value in EVERY assignment.

### Important Logical Equivalences to MEMORISE

| # | Equivalence | Law Name |
|---|---|---|
| (a) | P∨q ≡ q∨P | ∨ is commutative |
| (b) | P∧q ≡ q∧P | ∧ is commutative |
| (c) | (P∨q)∨r ≡ P∨(q∨r) | ∨ is associative |

| | | |
|---|---|---|
| (d) | $(P \wedge q) \wedge r \equiv P \wedge (q \wedge r)$ | $\wedge$ is associative |
| (e) | $P \vee (q \wedge r) \equiv (P \vee q) \wedge (P \vee r)$ | $\vee$ distributes over $\wedge$ |
| (f) | $P \wedge (q \vee r) \equiv (P \wedge q) \vee (P \wedge r)$ | $\wedge$ distributes over $\vee$ |
| (g) | $F \vee P \equiv P$ | Identity law (F = False) |
| (h) | $T \wedge P \equiv P$ | Identity law (T = True) |
| (i) | $P \vee P \equiv P$ | Idempotent law for $\vee$ |
| (j) | $P \wedge P \equiv P$ | Idempotent law for $\wedge$ |
| (k) | $P \wedge (P \vee q) \equiv P$ | Absorption law |
| (l) | $P \vee (P \wedge q) \equiv P$ | Absorption law |
| (m) | $F \wedge P \equiv F$ | Domination law |
| (n) | $T \vee P \equiv T$ | Domination law |
| (o) | $\neg(P \vee q) \equiv \neg P \wedge \neg q$ | De Morgan's law |
| (p) | $\neg(P \wedge q) \equiv \neg P \vee \neg q$ | De Morgan's law |
| (q) | $P \rightarrow q \equiv \neg P \vee q$ | Implication law |
| (r) | $P \rightarrow q \equiv \neg q \rightarrow \neg P$ | Contrapositive law |

| | | |
|---|---|---|
| (s) | $P \vee \neg P \equiv T$ | Tautology |
| (t) | $P \wedge \neg P \equiv F$ | Contradiction |
| (u) | $(P \rightarrow q) \wedge (q \rightarrow P) \equiv P \leftrightarrow q$ | Equivalence |

### Worked Equivalence Example 1 (from handout): Show $(p \rightarrow q) \wedge (q \rightarrow p) \equiv p \leftrightarrow q$

**Method – Truth Table**

| p | q | p→q | q→p | (p→q)∧(q→p) | p↔q |
|---|---|---|---|---|---|
| T | T | T | T | T | T |
| T | F | F | T | F | F |
| F | T | T | F | F | F |
| F | F | T | T | T | T |

Columns 5 and 6 are identical → $(p \rightarrow q) \wedge (q \rightarrow p) \equiv p \leftrightarrow q$ ✔ ■

### Worked Equivalence Example 2 (from handout): Show $(p \wedge q) \rightarrow p$ is a tautology

**Algebraic Proof**

$(p \wedge q) \rightarrow p$

$\equiv \neg(p \wedge q) \vee p$ [implication law: $A \rightarrow B \equiv \neg A \vee B$]

$\equiv (\neg p \vee \neg q) \vee p$ [De Morgan]

$\equiv (\neg q \vee \neg p) \vee p$ [commutative]

$\equiv \neg q \vee (\neg p \vee p)$ [associative]

$\equiv \neg q \vee T$ [$\neg p \vee p = T$]

$\equiv T$ [domination: $T \vee$ anything $= T$]

$\therefore (p \wedge q) \rightarrow p$ is a TAUTOLOGY. ■

### Worked Equivalence Example 3 (from handout): Show $(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$

**Algebraic Proof (Contrapositive)**

$(\neg q \rightarrow \neg p)$

$\equiv \neg(\neg q) \vee (\neg p)$ [implication law]

$\equiv q \vee (\neg p)$ [double negation: $\neg\neg q = q$]

$\equiv \neg p \vee q$ [commutative]

$\equiv p \rightarrow q$ [implication law: $\neg p \vee q = p \rightarrow q$]

$\therefore (p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$ (Contrapositive Law) ■

## 3.6 Conjunctive Normal Form (CNF)

**Literal:** An atom p or its negation ¬p.

**Clause:** A disjunction of one or more literals: l■ ∨ l■ ∨ … ∨ l■

**CNF (Conjunctive Normal Form):** A conjunction of clauses: (c■) ∧ (c■) ∧ … ∧ (c■), where each c■ is a clause.

### Examples of Clauses

- P (single literal)
- ¬p
- p ∨ q
- p ∨ ¬r ∨ ¬s

### CNF vs. Non-CNF

IN CNF: (¬p∨q∨r) ∧ (¬q∨r) ∧ (¬r)

NOT in CNF: (¬p∨q∨r) ∧ ((p∧¬q)∨r) ∧ (¬r) [middle clause is not a disjunction of literals]

NOT in CNF: (¬p∨q∨r) ∧ ¬(¬q∨r) ∧ (¬r) [¬(...) inside is not a literal]

**Transformation Algorithm to CNF:**

Step 1: Eliminate implications: A→B ≡ ¬A ∨ B

Step 2: Push negations inward using De Morgan: ¬(A∧B) ≡ ¬A∨¬B and ¬(A∨B) ≡ ¬A∧¬B

Step 3: Eliminate double negations: ¬¬A ≡ A

Step 4: Distribute ∨ over ∧: A∨(B∧C) ≡ (A∨B)∧(A∨C)

### Worked CNF Example (from handout): Convert (¬p→¬q)→(p→q) to CNF

#### Solution – Step by Step

Starting: (¬p→¬q) → (p→q)

Step 1 – Eliminate implications (A→B ≡ ¬A∨B):

Inner: ¬p→¬q ≡ ¬(¬p)∨¬q = p∨¬q

Inner: p→q ≡ ¬p∨q

Outer: (p∨¬q) → (¬p∨q)

≡ ¬(p∨¬q) ∨ (¬p∨q) [implication law]

Step 2 – Push negation inward:

¬(p∨¬q) ≡ ¬p ∧ ¬(¬q) = ¬p ∧ q [De Morgan]

So: (¬p ∧ q) ∨ (¬p ∨ q)

Step 3 – No double negations remain.

Step 4 – Distribute ∨ over ∧:

```
(¬p ∧ q) ∨ (¬p ∨ q)

= (¬p ∨ (¬p ∨ q)) ∧ (q ∨ (¬p ∨ q)) [A∨(B∧C) = (A∨B)∧(A∨C)]

= (¬p ∨ ¬p ∨ q) ∧ (q ∨ ¬p ∨ q) [associative]

= (¬p ∨ q) ∧ (¬p ∨ q) [idempotent: ¬p∨¬p=¬p, q∨q=q]

= (¬p ∨ q) [idempotent on the whole clause]

CNF Result: (¬p ∨ q) ✔

Note: This is also equivalent to p → q, confirming the formula simplifies nicely.
```

### Validity Testing with CNF

A formula in CNF is **valid (tautology)** if every clause contains both a literal and its negation (i.e. contains L■ ∨ ¬L■ for some L■).

**Validity Examples**

```
1. ¬p ∨ q ∨ p ∨ r → VALID (contains p and ¬p)

2. (¬p ∨ q ∨ p) ∧ (r ∨ ¬r) → VALID (each clause has complement pair)

3. (¬p ∨ q ∨ p) ∧ (r ∨ s) → NOT VALID (second clause has no complement pair)
```

## 3.7 Natural Deduction

**Natural deduction** is used to prove the validity of a sequent Γ ■ S.

We assume all formulas in Γ are true, then apply rules to derive S.

The goal is to show: *if the premises are true, the conclusion must also be true*.

### Modus Ponens (MP / E⇒)

If A⇒B and A are both true, then B is true.

**Rule: Modus Ponens (MP / E⇒)**

```
m A ⇒ B

n A

■■■■■■■■■

B E⇒ m,n
```

### Modus Tollens (MT)

If A→B and ¬B, then ¬A.

**Rule: Modus Tollens (MT)**

```
n A → B

m ¬B

■■■■■■■■■

¬A MT n,m
```

### Hypothetical Syllogism (HS)

If A→B and B→C, then A→C. (Transitivity of implication / chain rule)

**Rule: Hypothetical Syllogism (HS)**

```
n A → B

m B → C

■■■■■■■■■

A → C HS n,m
```

### Disjunctive Syllogism (DS)

If A∨B and ¬A, then B.

**Rule: Disjunctive Syllogism (DS)**

```
n A ∨ B

m ¬A

■■■■■■■■■

B DS n,m
```

### Disjunction Introduction (∨I)

If A is true, then A∨B is true (for ANY B).

**Rule: Disjunction Introduction (∨I)**

```
n A

■■■■■■■■■

A ∨ B ∨I n
```

### Disjunction Elimination (∨E)

If P→Q and R→Q and P∨R, then Q.

**Rule: Disjunction Elimination (∨E)**

```
n1 P → Q

n2 R → Q

n3 P ∨ R

■■■■■■■■■

Q ∨E
```

### Negation Introduction (¬I / Reductio ad Absurdum)

Assume A; if you derive both B and ¬B (contradiction), then ¬A.

**Rule: Negation Introduction (¬I / Reductio ad Absurdum)**

```
m | A H (hypothesis)

n | B

p | ¬B I¬ m,n,p

■■■■■■■■■■■■■

¬A
```

### Negation Elimination (¬E / Double Negation)

¬¬A implies A.

**Rule: Negation Elimination (¬E / Double Negation)**

```
n ¬¬A

■■■■■■■■■

A E¬ n
```

### Worked Natural Deduction Example 1 (from handout): Prove B from A∨B, ¬A

**Proof**

```
Given: A∨B, ¬A Goal: B

1. A ∨ B (Premise)

2. ¬A (Premise)

3. B DS 1,2 [Disjunctive Syllogism: A∨B and ¬A → B]

∴ B is proven. ■
```

### Worked Natural Deduction Example 2 (Extra): Prove ¬A from A→B, ¬B

**Proof**

```
Given: A→B, ¬B Goal: ¬A

1. A → B (Premise)

2. ¬B (Premise)

3. ¬A MT 1,2 [Modus Tollens]

∴ ¬A is proven. ■
```

### Worked Natural Deduction Example 3 (Extra): Prove A→C from A→B, B→C

**Proof**

```
Given: A→B, B→C Goal: A→C

1. A → B (Premise)

2. B → C (Premise)

3. A → C HS 1,2 [Hypothetical Syllogism]

∴ A→C is proven. ■
```

## 3.8 Limitations of Propositional Logic

In propositional logic, the **internal structure** of basic propositions is hidden.

For example: 'John is a science student' is just p — we cannot reason about 'John' as an object or 'science student' as a property.

This motivates **Predicate Logic** (Chapter 4), which exposes that internal structure.

# CHAPTER 4: PREDICATE LOGIC

Predicate logic (First-Order Logic, FOL) extends propositional logic by exposing the **internal structure** of propositions: objects, properties, and relations.

It adds **quantifiers** ∀ (for all) and ∃ (there exists) to express statements about quantities of objects.

Predicate logic incorporates all propositional connectives: ∨, ∧, →, ↔, plus quantifiers.

**Constant:** A 'proper name' for a specific object: a, b, c, j (for John), m (for Mary), …

**Variable:** Represents indefinite objects: x, y, z, …

**Predicate (n-place):** A property or relation applied to n objects. Written as a capital letter.

**Types of predicates:**

• 1-place (unary): Wx = 'x walks'

• 2-place (binary): Sxy = 'x sees y'

• 3-place (ternary): Gxyz = 'x gives y to z'

## 4.1 Translation Key & Examples

Choose a key that maps natural language expressions to logical letters and STICK TO IT.

**Translation Table (from handout)**

```
Natural Language → Logical Formula
```

■■■

```
Natural: John likes someone

Step 1: Ljx ('John likes x')

Step 2: ∃xLjx ('there exists an x that John likes')

Natural: Everyone liked the toad (t = toad)

Step 1: Lxt

Step 2: ∀xLxt ('for every x, x liked the toad')

Natural: Nigel is taller than everyone (n = Nigel)

Step 1: Tnx

Step 2: ∀xTnx ('for all x, Nigel is taller than x')
```

**✔ Universals go with CONDITIONALS: 'All A are B' → ∀x(Ax → Bx)**
**✔ Existentials go with CONJUNCTIONS: 'Some A are B' → ∃x(Ax ∧ Bx)**

## 4.3 Syntax of Quantifiers

A quantifier must always be followed by one or more variables and then a formula.

Complex formulas (with connectives and nested quantifiers) can appear inside quantifiers.

**Nested Quantifier Examples**

```
∃x (Fx → Fa)

∃y∀x ((Rxy & Ryx) → (Lxy & Lyx))
```

### Universe of Discourse

Every quantifier implicitly ranges over a **domain (universe of discourse)** D.

Example: ∀xHx for 'everyone is happy' implicitly assumes x ranges over people.

If D = {all things}, then 'everyone is happy' must be written as ∀x(Px → Hx) (for all x, if x is a person, then x is happy).

## 4.4 Syllogistic Statements

A syllogism is a logical argument with two premises and a conclusion.

**Classic Syllogism (from handout)**

```
Major premise: All men are mortal

Minor premise: Aristotle is a man

Conclusion: Aristotle is mortal

In predicate logic:

∀x(Mx → Rx) (all men are mortal: if x is a man, x is mortal)

Ma (Aristotle is a man)

■■■■■■■■■■■■■■■

Ra (Aristotle is mortal) – by Universal Instantiation + Modus Ponens
```

**Standard syllogistic forms:**

### Syllogistic Forms in Predicate Logic

'All A are B' → ∀x(Ax → Bx)

'Some A are B' → ∃x(Ax ∧ Bx)

'No A are B' → ¬∃x(Ax ∧ Bx) or equivalently ∀x(Ax → ¬Bx)

'Some A are not B' → ∃x(Ax ∧ ¬Bx)

WARNING: 'All A are B' is NOT ∀x(Ax ∧ Bx)

That would mean 'Everything is both A and B' — incorrect!

WARNING: 'Some A are B' is NOT ∃x(Ax → Bx)

That would be satisfied even if no A exists — incorrect!

## 4.5 Conjunctive Predicate Combinations

When an adjective + noun combination genuinely means BOTH properties, use conjunction.

### Conjunctive Combination Examples (from handout)

(a) Every Purple Squid is Friendly

→ ∀x((Px ∧ Sx) → Fx)

Read: For all x, if x is Purple AND x is a Squid, then x is Friendly.

(b) Some Monsters are Hairy Drunkards

→ ∃x(Mx ∧ (Hx ∧ Dx))

Read: There exists x such that x is a Monster AND Hairy AND a Drunkard.

(c) No Invisible Lemur is a Criminal

→ ¬∃x((Ix ∧ Lx) ∧ Cx)

Read: There is no x that is Invisible AND a Lemur AND a Criminal.

- Some compound nouns CANNOT be split: 'alleged criminal', 'expectant mother', 'deer hunter' — treat these as SINGLE predicates.
- 'Bostonian cabdriver' could mean (Bx∧Cx) or a single predicate depending on context.

## 4.6 Disjunctive Predicate Combinations

Surface conjunctions ('and') in natural language can be LOGICAL DISJUNCTIONS.

### Worked Example: 'All cats and dogs make suitable pets'

First attempt (WRONG):

∀x((Cx ∧ Dx) → Sx) — this says 'everything that is BOTH a cat AND a dog'

But nothing is both a cat and a dog! This is always vacuously true. ✗

Correct Translation 1 (conjunction of two universals):

∀x(Cx → Sx) ∧ ∀x(Dx → Sx)

Correct Translation 2 (disjunction in antecedent):

∀x((Cx ∨ Dx) → Sx)

Read: For all x, if x is a cat OR a dog, then x makes a suitable pet. ✔

## 4.7 Laws and Rules of Inference in Predicate Logic

We need additional laws to reason with quantifiers.

### Laws of Quantifier Distribution

**Law QD1: ¬(∀x)ϕ(x) ■ (∃x)¬ϕ(x)**

If ϕ is not universally true, there exists an x for which it fails.

**Law QD2: (∀x)(ϕ(x)∧ψ(x)) ■ (∀x)ϕ(x)∧(∀x)ψ(x)**

∀ distributes over ∧ (both ways).

**Law QD3: (∃x)(ϕ(x)∨ψ(x)) ■ (∃x)ϕ(x)∨(∃x)ψ(x)**

∃ distributes over ∨ (both ways).

**Law QD4: (∀x)ϕ(x)∨(∀x)ψ(x) ■ (∀x)(ϕ(x)∨ψ(x))**

∀ can be raised OUT of a disjunction (one direction only — implication, not equivalence).

**Law QD5: (∃x)(ϕ(x)∧ψ(x)) ■ (∃x)ϕ(x)∧(∃x)ψ(x)**

∃ can be lowered INTO a conjunction (one direction only).

### Laws of Quantifier Scope

**Law QS6: (∀x)(∀y)ϕ(x,y) ■ (∀y)(∀x)ϕ(x,y)**

Order of two ∀ quantifiers is irrelevant.

**Law QS7: (∃x)(∃y)ϕ(x,y) ■ (∃y)(∃x)ϕ(x,y)**

Order of two ∃ quantifiers is irrelevant.

**Law QS8: (∃x)(∀y)ϕ(x,y) ■ (∀y)(∃x)ϕ(x,y)**

∃ before ∀ implies ∀ before ∃ (NOT vice versa).

■ QS8 is NOT an equivalence! '∃x∀y' is STRONGER than '∀y∃x'.

### Four Rules of Inference for Quantifiers

### Universal Instantiation (U.I.)

From ∀xϕ(x), conclude ϕ(c) for any constant c.

**Rule: Universal Instantiation (U.I.)**

(∀x)ϕ(x)

∴ ϕ(c) [for any constant c]

Intuition: if ϕ is true of EVERYTHING, it is true of any specific thing.

### Universal Generalization (U.G.)

From $\phi(v)$ for ARBITRARY v, conclude $\forall x\phi(x)$.

**Rule: Universal Generalization (U.G.)**

$\phi(v)$

$\therefore (\forall x)\phi(x)$ [v must be a completely arbitrary constant]

Intuition: if $\phi$ holds for an arbitrary individual, it holds for ALL individuals.

### Existential Generalization (E.G.)

From $\phi(c)$ for some specific c, conclude $\exists x\phi(x)$.

**Rule: Existential Generalization (E.G.)**

$\phi(c)$

$\therefore (\exists x)\phi(x)$

Intuition: if $\phi$ holds for one specific thing, at least one thing satisfies $\phi$.

### Existential Instantiation (E.I.)

From $\exists x\phi(x)$, introduce a NEW constant w: $\phi(w)$.

**Rule: Existential Instantiation (E.I.)**

$(\exists x)\phi(x)$

$\therefore \phi(w)$ [w is a brand new constant, never used before]

Intuition: name the thing that satisfies $\phi$, as long as we haven't named it yet.

### Worked Proof 1 (U.I. example from handout): Prove H(a) from G(a) and $\forall x(G(x)\rightarrow H(x))$

**Proof**

1. $(\forall x)(G(x)\rightarrow H(x))$ Given
2. $G(a)$ Given
3. $G(a)\rightarrow H(a)$ 1, U.I. [instantiate x to constant a]
4. $H(a)$ 2,3, M.P. [Modus Ponens]

$\therefore$ H(a) is proven. ■

### Worked Proof 2 (U.G. example from handout): Prove $\forall x(R(x)\rightarrow W(x))$ from $\forall x(R(x)\rightarrow Q(x))$ and $\forall x(Q(x)\rightarrow W(x))$

**Proof**

1. $(\forall x)(R(x)\rightarrow Q(x))$ Given
2. $(\forall x)(Q(x)\rightarrow W(x))$ Given
3. $R(v)\rightarrow Q(v)$ 1, U.I. [arbitrary constant v]
4. $Q(v)\rightarrow W(v)$ 2, U.I.
5. $R(v)\rightarrow W(v)$ 3,4, H.S. [Hypothetical Syllogism]
6. $(\forall x)(R(x)\rightarrow W(x))$ 5, U.G. [v was arbitrary $\rightarrow$ generalise]

∴ ∀x(R(x)→W(x)) is proven. ■

### Worked Proof 3 (E.I. example from handout): Prove (∃x)P(x)∧(∃x)Q(x) from ∃x(P(x)∧Q(x))

**Proof**

1. (∃x)(P(x)∧Q(x)) Given
2. P(w)∧Q(w) 1, E.I. [w is a brand new constant]
3. P(w) 2, Simplification
4. (∃x)P(x) 3, E.G.
5. Q(w) 2, Simplification
6. (∃x)Q(x) 5, E.G.
7. (∃x)P(x)∧(∃x)Q(x) 4,6, Conjunction

∴ The result is proven. ■

## 4.8 Indirect Proof

Also called **Reductio ad Absurdum** or **Proof by Contradiction**.

Strategy: ASSUME the NEGATION of what you want to prove, derive a contradiction (A∧¬A), conclude the original statement must be true.

### Worked Indirect Proof (from handout): Prove (∀x)G(x)→(∃x)G(x) from no assumptions

**Proof by Contradiction**

1. ¬((∀x)G(x)→(∃x)G(x)) Auxiliary Premise (assume negation of goal)
2. ¬(¬(∀x)G(x)∨(∃x)G(x)) 1, Implication law: A→B ≡ ¬A∨B
3. (∀x)G(x) ∧ ¬(∃x)G(x) 2, De Morgan + Double Negation
4. (∀x)G(x) 3, Simplification
5. ¬(∃x)G(x) 3, Simplification
6. (∀x)¬G(x) 5, QD1 [¬∃x ≡ ∀x¬]
7. G(a) 4, U.I. [instantiate to constant a]
8. ¬G(a) 6, U.I.
9. G(a) ∧ ¬G(a) 7,8, Conjunction ← CONTRADICTION
10. (∀x)G(x)→(∃x)G(x) 1-9, Indirect Proof ■

## 4.9 Conditional Proof

Strategy: ASSUME the antecedent as an auxiliary premise, then DERIVE the consequent. The whole conditional is then established by the Conditional Proof rule.

### Worked Conditional Proof (from handout): Prove (∀x)G(x)→(∃x)G(x)

**Proof by Conditional Proof**

1. (∀x)G(x) Auxiliary Premise (assume antecedent)
2. G(a) 1, U.I. [instantiate to constant a]

```
3. (∃x)G(x) 2, E.G. [generalise existentially]

4. (∀x)G(x)→(∃x)G(x) 1-3, Conditional Proof

We assumed (∀x)G(x) in line 1, derived (∃x)G(x) in line 3,

so the conditional holds. ■
```

■ *Compare Indirect Proof (4.8) and Conditional Proof (4.9): both prove the same result but by different strategies. Conditional Proof is simpler here — assume antecedent, derive consequent.*

## EXAM QUICK-REFERENCE SUMMARY

| Topic | Key Fact |
|---|---|
| Mathematical Induction | 3 steps: Base Case → IH (assume k) → Show k+1. Only for integer domains. |
| Structural Induction | 2 steps: Base Case → Combination step (assume holds for parts, prove for whole). |
| Boolean Simplification | Apply postulates step by step. Use K-maps for up to 4-6 variables. |
| K-Map grouping | Groups must be rectangular, size = power of 2. Larger group = fewer literals. |
| Truth table size | n variables → 2■ rows. |
| Tautology vs Contradiction | Tautology: always T. Contradiction: always F. Contingency: sometimes T, sometimes F. |
| CNF Transformation | Eliminate →, push ¬ inward (De Morgan), eliminate ¬¬, distribute ∨ over ∧. |
| Natural Deduction key rules | MP (A→B, A ■ B), MT (A→B, ¬B ■ ¬A), HS (A→B, B→C ■ A→C), DS (A∨B, ¬A ■ B). |
| Quantifier rules | U.I.: ∀xϕ → ϕ(c). U.G.: ϕ(v) → ∀xϕ. E.G.: ϕ(c) → ∃xϕ. E.I.: ∃xϕ → ϕ(w, new). |
| Universals with conditionals | 'All A are B' → ∀x(Ax→Bx) NOT ∀x(Ax∧Bx). |
| Existentials with conjunctions | 'Some A are B' → ∃x(Ax∧Bx) NOT ∃x(Ax→Bx). |
| QD1 (important!) | ¬∀xϕ(x) ■ ∃x¬ϕ(x) and ¬∃xϕ(x) ■ ∀x¬ϕ(x). |

**Good luck in your exams! Study every worked example above.**